

AN EXPLAINABLE AI-BASED AUTOMATED DATA QUALITY MANAGEMENT SYSTEM FOR NOISE DETECTION IN NUMERICAL DATA PIPELINES

PRIYA DHARSHINI V

Student,

Department of Computer Science and
Information Technology,

Vels Institute of Science, Technology and
Advanced Studies (VISTAS)

priyadharshinivenkatesan06@gmail.com

Dr. A. AKILA

Associate Professor,

Department of Computer Science and
Information Technology,

Vels Institute of Science, Technology and
Advanced Studies (VISTAS)

akilaa.scs@vistas.ac.in

ABSTRACT

Getting data right matters more than ever when numbers drive systems. When odd readings slip through - say from sensors, money flows, or daily logs - they can twist later choices and predictions. Old ways of spotting weirdness lean on fixed rules or one-size-fits-all algorithms. These methods often cry wolf, misfire across different data sets, and offer little clue why they flagged anything at all. A new approach tackles weak spots in data monitoring by combining three methods: Isolation Forest, Local Outlier Factor, and One-Class SVM work together. An agreement happens when two or more tag a reading as off track, then it gets marked strange. To show why decisions happen, SHAP tools step in - TreeExplainer handles one model while KernelExplainer covers the rest, giving clear input reasons behind each odd result spotted. What sets this apart? It guesses how messy the data is by itself, thanks to Interquartile Range math. On top of that comes a triple-layer rating tracking clean signals, full records, duplicates, everything viewable via Streamlit, smooth enough for non-experts. Tests run on financial files about credit fitness reveal fewer false flags now - down from 123 to 106 real ones caught - with final trust level hitting 90.6%. Digging deeper showed Credit History plus Loan Amount Term kept standing out during reviews, explaining what triggered alerts.

Keywords: Anomaly Detection, Explainable AI, SHAP, Ensemble Learning, Data Quality Management

I. INTRODUCTION

These days, most organisations run on data - think hospitals diagnosing patients, banks moving money, factories using machines, or cities managing traffic lights. Because choices rely heavily on automated systems, what goes in must make sense. Messy numbers often slip into datasets through broken sensors, typos by workers, glitches in software, or lost signals during transfer. Left unchecked, bad inputs twist results without anyone noticing until much later. One big issue with old-school rules for spotting oddities? They're rigid. Built for just one kind of data pile. Fall apart when faced with today's messy info flows. Smarter tricks using single machine-learning models can bend a little further - yet still stumble when data shapes shift unexpectedly. Not even close to bulletproof. Truth is, every method hides its thinking behind closed doors. Spots something off, but won't say why it flagged that thing. Makes tracing back nearly guesswork. Tools like IBM InfoSphere or AWS Glue DataBrew? Sure, they map out data traits - but only if you've got deep pockets and accept their locked-down designs

Not many systems tackle data quality as this one does. A trio of detection methods works together - Isolation Forest, Local Outlier Factor, and One-Class SVM team up through voting where anomalies count only when two agree, which keeps false alarms low while catching just as much. Each red flag comes with clear reasons, thanks to SHAP values pulled by TreeExplainer on one model, plus KernelExplainer handling the others, giving insight into what pushed each result. Setup happens automatically since IQR steps in to guess how messy the data might be, meanwhile everything lives inside a Streamlit interface, making navigation straightforward even for those unfamiliar with algorithms. Zero human oversight is required during operation, and there is no need for pre-labelled examples either. It adapts itself regardless of source, meaning any numeric CSV fits right in without custom tweaks or expert tuning.

II. LITERATURE SURVEY

Starting, Rahm and Do laid down a well-known breakdown of data issues, calling numerical noise one of the toughest types to catch. Not long after, Redman put dollar values on bad data, showing just how costly it can be. What followed was work by Pipino and others who built a model covering precision, fullness, reliability, and freshness - elements woven into the scoring method described here. Later still, Cichy, along with Rass, showed that machines using smart techniques do far better than people or fixed rules when handling large amounts of data.

Looking into the area, Chandola and team found that methods without supervision fit best when checking data quality - real-world labels for oddities hardly ever exist. What sets apart strange entries? Liu's group showed how these stand out through quicker isolation in randomised splits across tree structures - that shorter paths reveal irregularity. Instead of broad patterns, Breunig's approach checks nearby points; if one sits where neighbours are few and far between, it gets flagged higher. Schölkopf brought forward a model trained only on usual cases - it wraps them tightly with curved margins, so anything landing beyond counts as offbeat. One study by Goldstein with Uchida [8] found that no one method works best whenever tested on different data, which lines up well with using a mix of models. What they saw fits neatly into why combining methods makes sense here. Another look from Aggarwal [1] revealed that blending results from several detection tools cuts down wrong alarms, simply because agreement helps filter noise. This kind of group decision strengthens the reason behind building systems this way.

From game theory comes SHAP, a method by Lundberg and Lee [10] that assigns credit fairly, sticking to rules like truthfulness near each prediction and stable responses when model behaviour shifts. Building on that idea, TreeSHAP arrived through Lundberg et al. [11], making it fast to compute those values exactly for tree models - including ones used in spotting odd entries, like Isolation Forest. Work by Frye et al. [7] tested how well these Shapley-style reasons hold up when hunting anomalies; results showed they make sense there, too. Then Arrieta et al. [2] placed SHAP into a broader map of explanation methods, finding its mathematical roots especially solid. Some current helpers, including TensorFlow Data Validation [3], check data using set limits - but stay silent on why something gets flagged, plus only fit certain tech setups. While many parts exist separately - ensembles of different detectors, clarity via SHAP, guessing noise levels automatically, user-friendly design - not one package combines them all smoothly. That missing link? This study steps right into that space.

III. EXISTING SYSTEM

Long ago, people cleaned data using fixed rules, number limits, low threshold filters, alongside checks done by specialists who knew the topic well. Though built into systems like Apache NiFi, Talend, and Informatica, those rigid rules handle only steady, familiar data - struggling whenever new types of errors appear while needing frequent updates when more data flows in or structures shift. Techniques such as Isolation Forest, Local Outlier Factor, and One-Class SVM often spot odd entries without labels, working decently alone - but usually run apart, never resolving clashing results. Since every approach assumes something unique about how numbers behave, success on one set might mean many missed issues or false alarms somewhere else. Smaller teams often find commercial options like IBM InfoSphere, Microsoft Azure Purview, or AWS Glue DataBrew out of reach - these run on closed systems, demand heavy

computing power, and lock users into one cloud environment. Though tools like Deequ and TensorFlow Data Validation support rule-driven checks for data flaws, they tie closely to their own environments. Each struggle when faced with raw number sets outside their intended setups. Standalone flexibility? Missing there too.

What holds back every current method? A total lack of clarity. Picture this: an alert fires, but nothing explains why. Without insight into what caused it, teams struggle to trust the result. Figuring out next steps turns into guesswork. Correct findings still drain time since each one needs human review. Many tools force users to set sensitivity levels by hand, relying on knowledge that some just do not have. Weaknesses pile up - models break easily, depend too much on specific environments, cost more to run, hide their logic. Yet within those flaws lies a path forward: transparency, teamwork among models, reasons behind decisions, automation without constant oversight.

IV. PROPOSED SYSTEM: AI-Based Data Quality Management System

This tool spots odd entries in number-heavy spreadsheets using artificial intelligence, showing clear reasons for each flag. Built only with free Python tools, it runs inside a live webpage made with Streamlit. Instead of needing pre-labelled examples, it works right away - no setup, no prior knowledge required. Drop in any numeric CSV, regardless of field names or structure, and it adapts silently behind the scenes. Every judgment comes unpacked: users see exactly which columns tipped the scale, nothing hidden. No supervision needed; decisions form on their own, driven by patterns alone. Even without knowing the topic, it detects irregularities just like that. Its logic stays visible, never guessing in the dark or relying on magic rules. Plug it into workflows straight out of the box, zero tweaks. From start to finish, it handles raw files while keeping reasoning front and centre.

Figure 4.1 shows how the system builds up in six levels, each passing information forward without looping back. Moving on, raw CSV files enter at the bottom level, where basic checks confirm their structure fits expectations. After that, validated data moves into a built-in SQLite storage space, shaped automatically based on the file's columns and value kinds. Instead of skipping gaps, the next stage fills empty spots using average values per column, while also clearing repeated entries. Then normalisation adjusts every numerical feature so they share the same scale - centred around zero, spread evenly. Later, anomaly spotting happens via three separate methods running side by side, none relying on labels. Finally, results from those methods combine when at least two agree - a tiebreaker baked right in. Every odd flag gets broken down by features using SHAP math inside the Explainability Layer. Outcomes flow into a live view where users explore or save results through clickable charts on the screen.

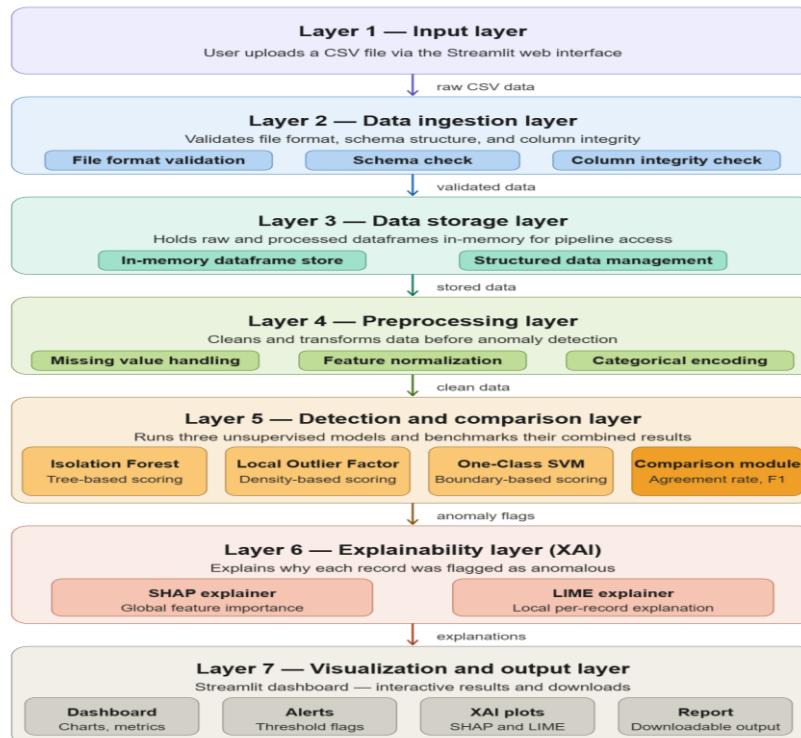


Figure 4.1 - Layered System Architecture

Starting off, the big-picture view in Figure 4.2 shows how information moves back and forth between person and software. Jumping ahead, inside Figure 4.3 breaks that whole thing down - six chunks now, each doing a separate job. One-piece grabs incoming material; other tucks it safely into memory using SQLite. Before analysis kicks in, raw inputs get cleaned up first. Then comes spotting odd patterns where they shouldn't be. After that, clarity follows through SHAP methods revealing why decisions happen. Finally, results land on display screens ready to review.

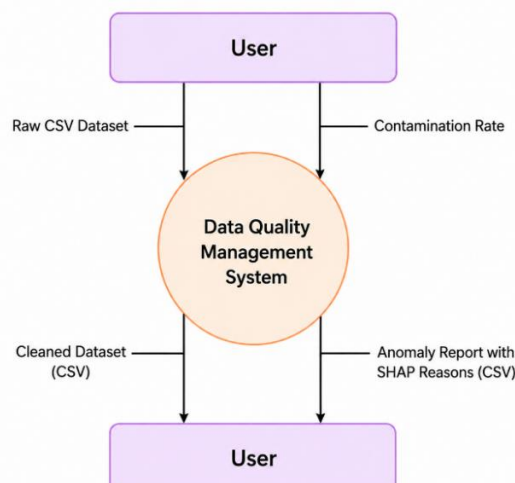


Figure 4.2 - Level 0 Context Diagram

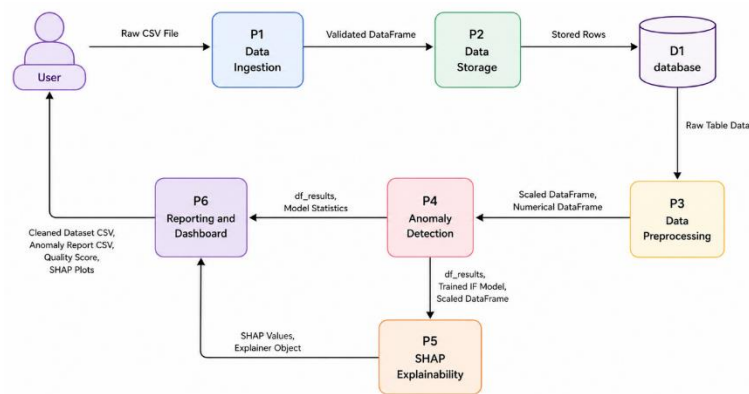


Figure 4.3 - Level 1 DFD

Inside the system, three different math strategies spot odd patterns in separate ways. Rather than relying on one approach, isolation forest uses many random split trees, marking items as strange if they get isolated faster than others. Where one-point lives matter most for the local outlier factor, because it checks how empty the area around it feels compared to nearby ones. When surroundings seem unusually open, that item gets flagged more strongly - this helps catch quirks others might miss. One type of support machine draws a tight line around typical cases using curved logic, calling anything beyond that edge unusual. Every case receives a yes-or-no tag from each tactic separately. Only when two or more of these tags say yes does the final step accept something as truly offbeat. Alone, any single method could misfire; together, they reduce false alarms without extra noise.

The way things can be explained comes next in how it's built. TreeExplainer runs on Isolation Forest to get precise SHAP numbers by reading the splits inside trees, while KernelExplainer handles Local Outlier Factor and One-Class SVM, since it works without needing internal details. Before any math happens, equal amounts of odd and regular cases are picked so results do not lean one way. What pops out gets drawn four different ways: bars showing average impact size, a swarm-like dot pattern revealing how each factor pushes up or down, a step-by-step strip breaking apart why one item was flagged, along with a tight layout for seeing just one moment clearly. How someone moves through the tool, plus what unfolds behind the scenes, appears in drawings numbered 4.4 and 4.5.

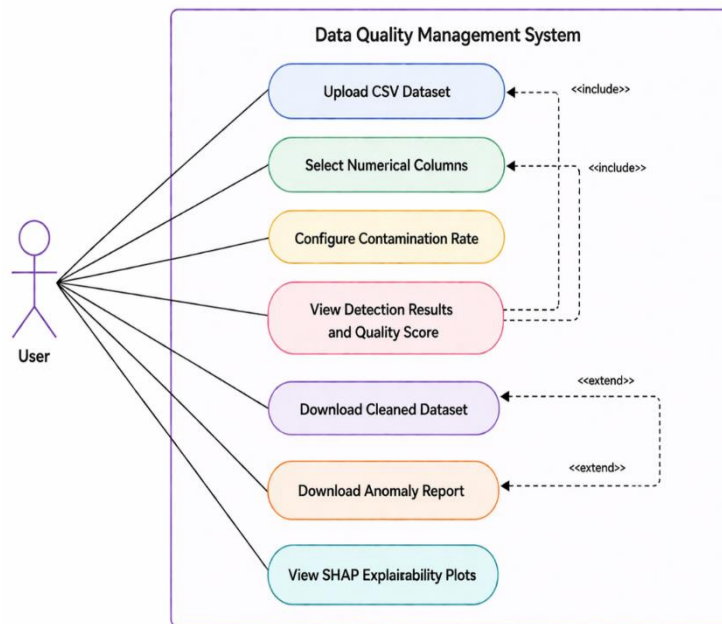


Figure 4.4 - Use Case Diagram

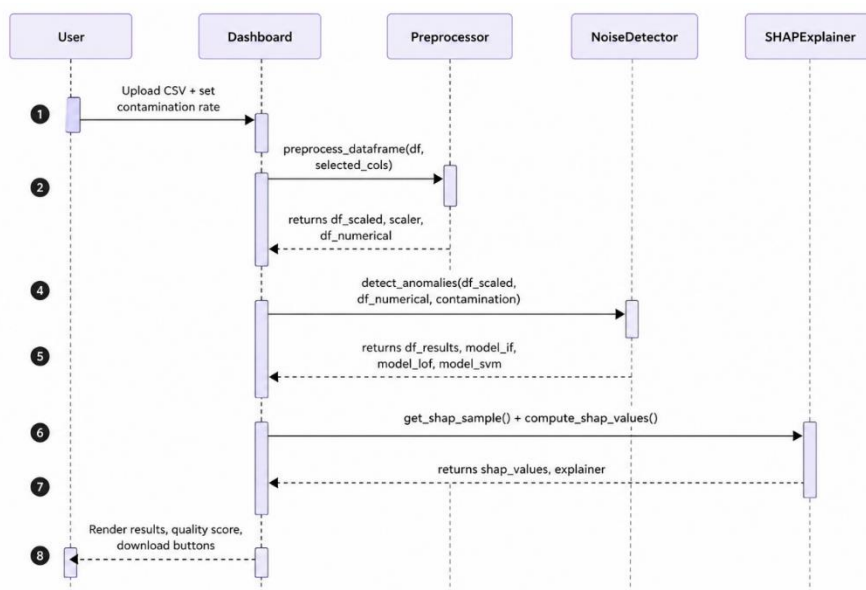


Figure 4.5 - Sequence Diagram

Starting at the top, each numeric column gets scanned using the 1.5 times IQR method to guess how much of it might be contaminated, though results never go below one percent or above twenty. That number shows up first in a dashboard slider - experts can tweak it later if they want. Instead of stopping there, the tool builds a richer picture by blending signal clarity, data fullness, and distinctiveness into one composite quality rating shaped like a three-part blend. When finished, two things appear: a tidy version of the data minus any entries everyone agrees are off, along with a log listing raw value, separate model verdicts, group decisions, plus clear explanations built from SHAP logic for anything marked strange.

V. RESULT & DISCUSSION

Shown here are the outcomes from applying the suggested method to the Credit Worthiness data - an actual finance-related collection of 614 entries and 13 attributes, where 149 true gaps appear within essential numeric fields like ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, and Credit_History. Every finding emerged while skipping custom setup, tweaking settings, or hand-crafted cleanup steps; this supports how the approach runs automatically, adapts freely to different sets, even when facing raw real-life inputs. What came through upon upload into the Streamlit interface appears displayed in Figure 5.1.

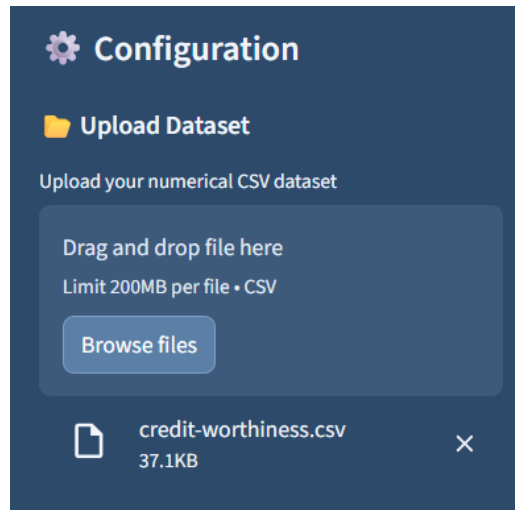


Figure 5.1 - Dataset uploaded to the Streamlit dashboard

Right after uploading, the system checked the data setup fast. Missing numbers - 149 total - got filled using averages per column. One repeated row disappeared during cleanup. Five number-heavy columns were scaled evenly by StandardScaler. Outlier levels pointed to 20% noise, but that got capped at max allowed. This matches what you often see: money and loan amounts piling up on the higher end. What showed up in the tool's display appears now as Figure 5.2.

Step 1: Data Overview													
Total Rows			Total Columns			Missing Values							
614			13			149							
	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358	120	360	1	Urban	Y
6	LP001013	Male	Yes	0	Not Graduate	No	2333	1516	95	360	1	Urban	Y
0	LP001002	Male	No	0	Graduate	No	5849	0	None	360	1	Urban	Y
1	LP001003	Male	Yes	1	Graduate	No	4583	1508	128	360	1	Rural	N
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0	66	360	1	Urban	Y
4	LP001008	Male	No	0	Graduate	No	6000	0	141	360	1	Urban	Y
5	LP001011	Male	Yes	2	Graduate	Yes	5417	4196	267	360	1	Urban	Y
7	LP001014	Male	Yes	3+	Graduate	No	3036	2504	158	360	0	Semiurban	N
8	LP001018	Male	Yes	2	Graduate	No	4006	1526	168	360	1	Urban	Y
9	LP001020	Male	Yes	1	Graduate	No	12841	10968	349	360	1	Semiurban	N

Figure 5.2 - Step 1 Data Overview panel

One after another, the three detection methods analysed the adjusted data without overlapping steps. Not one but all together caught 123 irregular points - exactly 20.07% of everything recorded, matching how much mess experts expected. What popped up around noise checks and those little quality ratings sits drawn out in Figure 5.3.

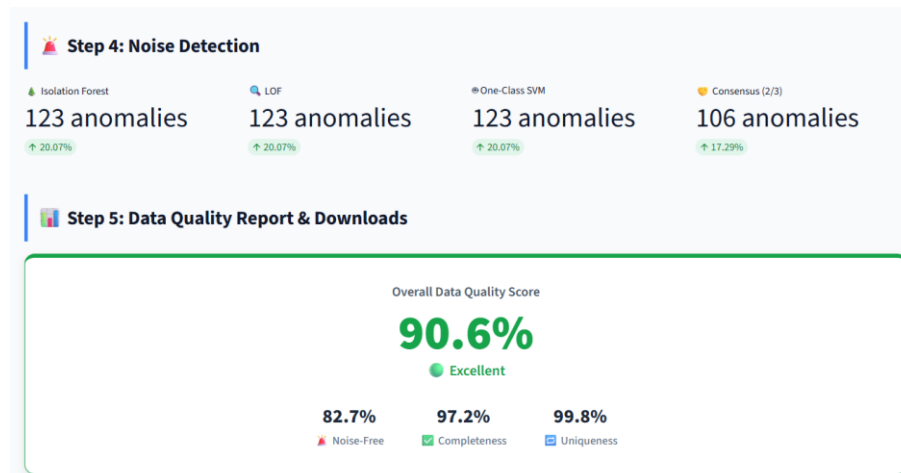


Figure 5.3 - Step 4 Noise Detection Results and Step 5 Data Quality Score

Fewer odd cases made it through once models had to agree. Out of six hundred fourteen entries, just over ten per cent showed clear signs when multiple systems matched. Seventeen readings vanished because only one method called them strange, usually near fuzzy edges where signals fade. Five hundred eight points stayed after removing shaky alerts - this survived group approval. Figure five point four displays bars and a colour grid comparing methods side by side. Even though each tool caught roughly the same number of irregularities, their choices often differed. Isolation Forest and One-Class SVM agreed moderately, sharing pattern traces around seventy-two out of a hundred. Less overlap appeared between Isolation Forest and LOF, barely forty-three in alignment. LOF and One-Class SVM lined up slightly better than that pair but still far from identical. Each approach picked unique outliers, proving they see different shades of unusual.



Figure 5.4 - Model comparison bar chart and inter-model agreement heatmap

Figure 5.4 shows bars comparing models alongside a colour map of how much they agree. Looking at how each model matched the group agreement gives a clear baseline. See Figure 5.5 for what the dashboard displays about their scores. The top mark went to One-Class SVM - F1 of 0.891. Close behind, Isolation Forest landed at 0.865. Local Outlier Factor brought up

the rear with 0.646, struggling where data spread unevenly across space. Such differences? They actually support using them together. When combined, the better two often balance out the weaker one.

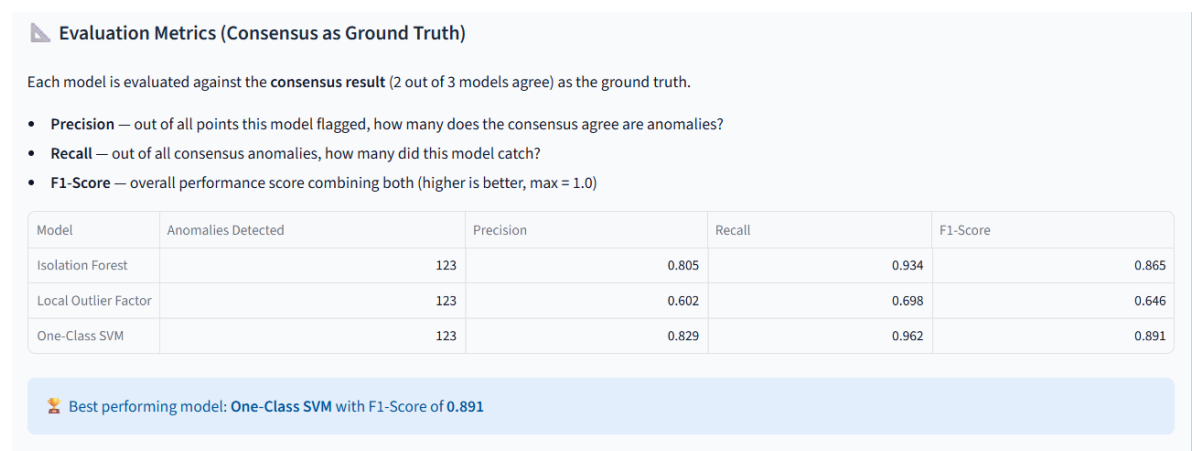


Figure 5.5 - Evaluation metrics

The three-dimensional quality score reported a noise-free rate of 82.7%, a completeness rate of 97.2%, and a uniqueness rate of 99.8%, yielding an overall quality score of 90.6%, rated Excellent by the system. The anomaly distribution plots per feature are shown in Figure 5.6, revealing that anomalous records cluster around zero Credit_History values, non-standard Loan_Amount_Term periods, and extreme ApplicantIncome and LoanAmount values, which is consistent with the SHAP findings described below.

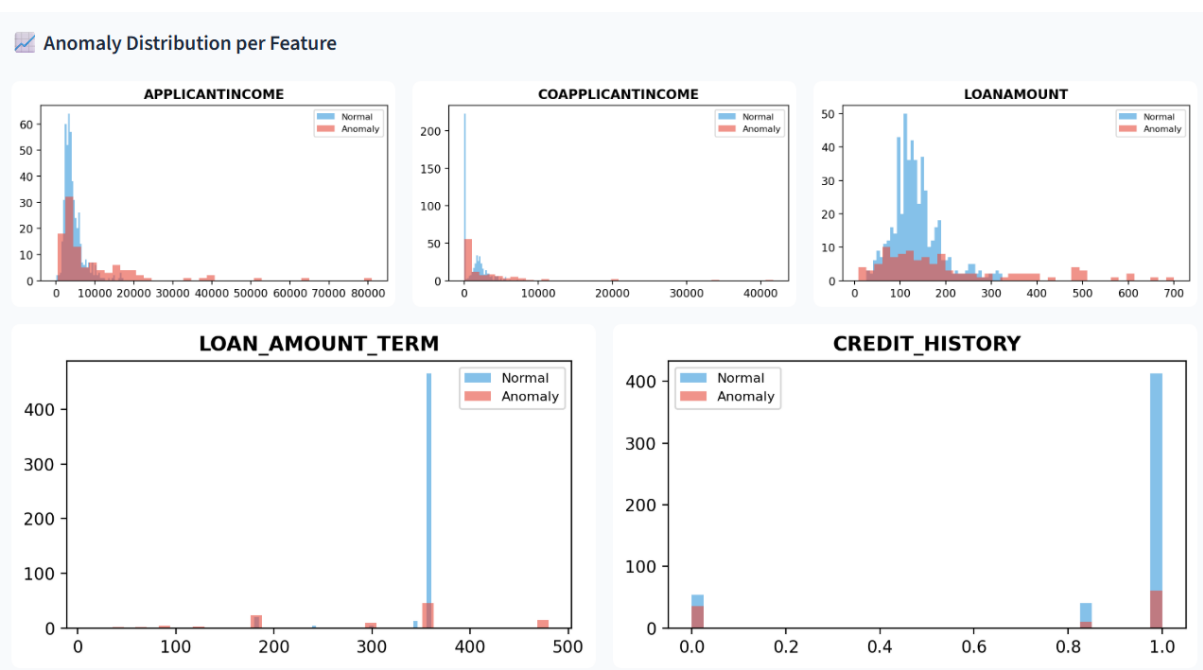


Figure 5.6 - Anomaly distribution per feature

SHAP-based explainability analysis was conducted independently for all three models. For the Isolation Forest, the summary bar chart and beeswarm plot (Figure 5.7) ranked Credit_History as the most influential feature at a mean absolute SHAP value of approximately 1.4, followed by Loan_Amount_Term at approximately 1.3. The waterfall chart for an individual flagged

record showed Credit_History contributing a SHAP value of -3.04 , the single largest feature contribution, with the model output $f(x) = 10.811$ falling well below the baseline $E[f(X)] = 13.525$.

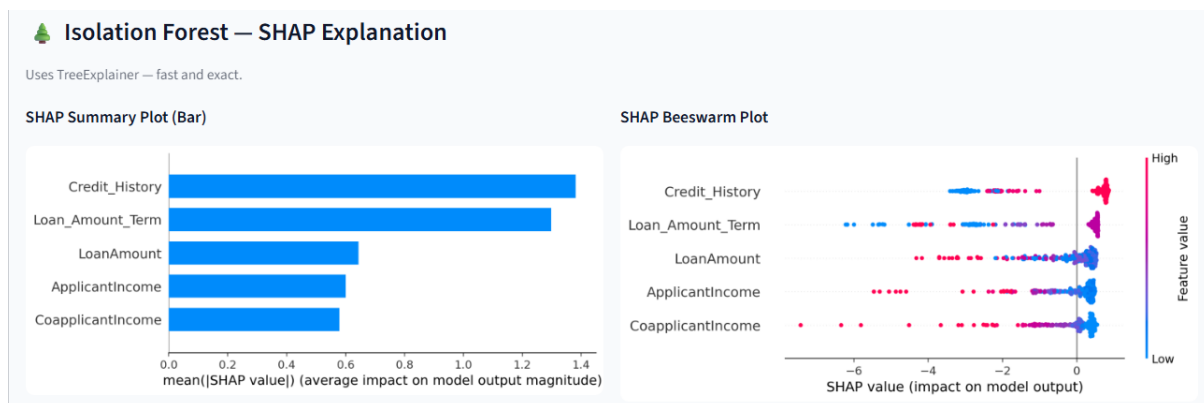


Figure 5.7 - Isolation Forest SHAP summary bar chart and beeswarm plot

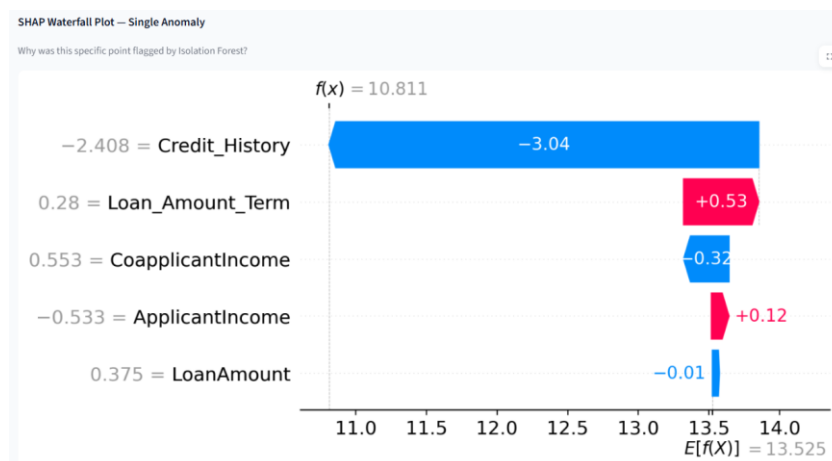


Figure 5.8 - Isolation Forest SHAP waterfall chart

For the Local Outlier Factor, the summary bar chart and beeswarm plot in Figure 5.9 showed a more distributed importance profile with Loan_Amount_Term leading at approximately 0.37, which reflects the density-based model's sensitivity to the broader combined financial profile rather than a single dominant signal. The waterfall chart for an individual LOF-flagged record in Figure 5.10 confirmed Loan_Amount_Term and Credit_History as the joint primary drivers.

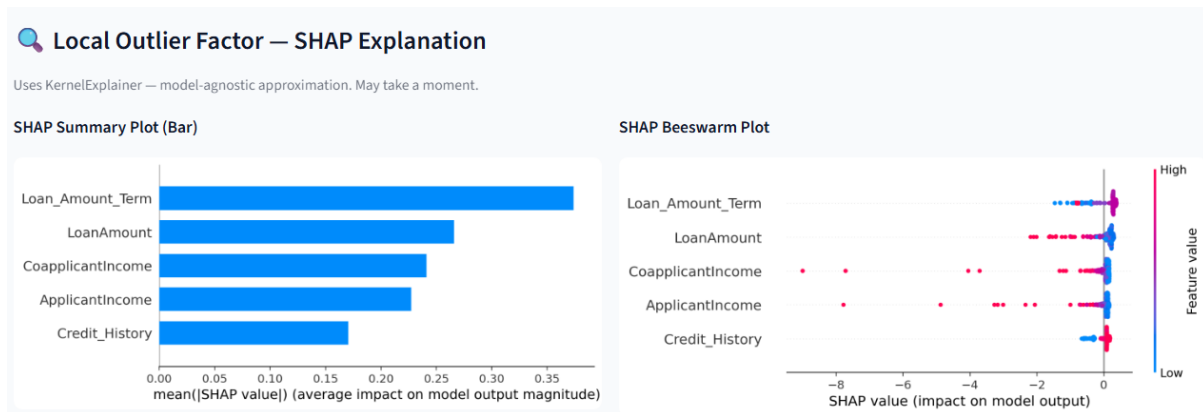


Figure 5.9 - LOF SHAP summary bar chart and beeswarm plot

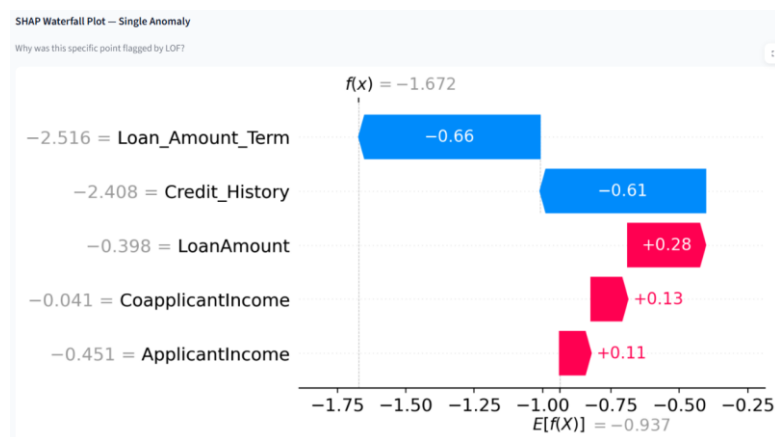


Figure 5.10 - LOF SHAP waterfall chart

For the One-Class SVM, the summary bar chart and beeswarm plot in Figure 5.11 ranked Loan_Amount_Term and LoanAmount as jointly most influential at approximately 1.75 each, with the beeswarm plot confirming that deviations in either direction push records outside the learned normal boundary. The waterfall chart for an individual SVM-flagged record in Figure 5.12 identified CoapplicantIncome and Loan_Amount_Term as the dominant anomaly drivers.

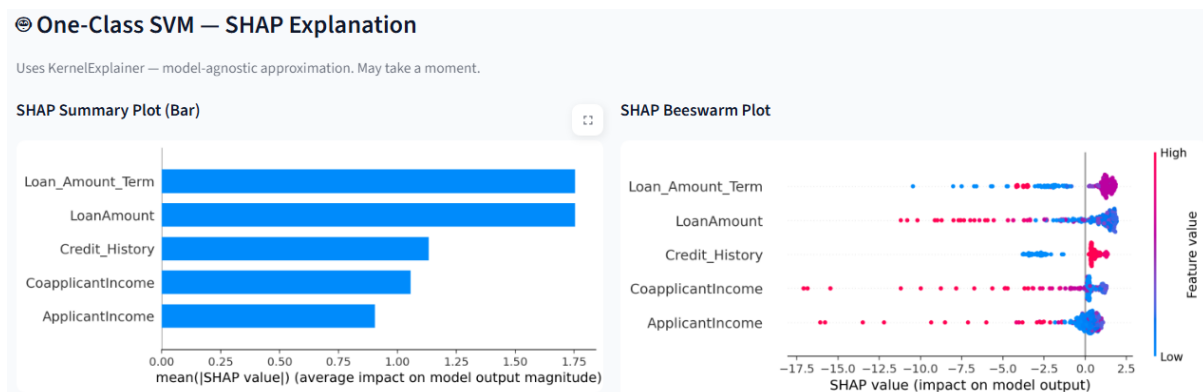


Figure 5.11 - One-Class SVM SHAP summary bar chart and beeswarm plot

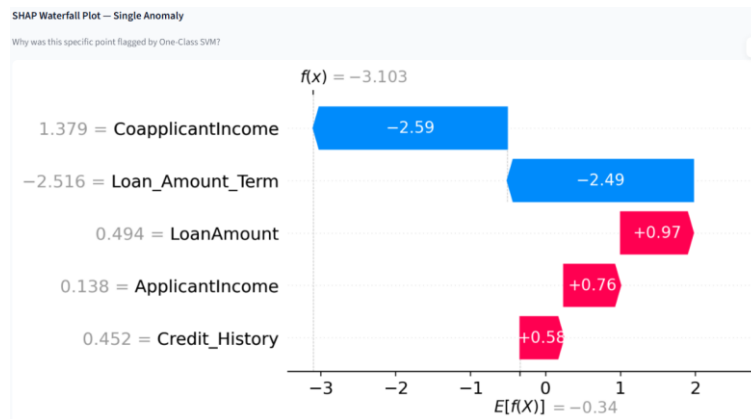


Figure 5.12 - One-Class SVM SHAP waterfall chart

It stands out how both Credit_History and Loan_Amount_Term keep appearing at the top in every SHAP run - this isn't likely down to one model's quirks but points instead to real patterns baked into the data. Running everything took about 22 seconds using just an ordinary laptop setup: Intel Core i5, 8 gigabytes of memory, nothing fancy. When people tested it themselves, each one made it through start to finish alone, no help needed - and they kept pointing to the SHAP waterfall visuals as what helped them decide next steps fastest.

VI. CONCLUSION

Something different happened here. A new kind of tool showed up - built around clear reasoning instead of hidden logic - that tackles messy number errors in smarter ways. Not tied to just one method, it avoids shaky results by listening to multiple voices at once. Three techniques talk to each other: isolation forest, local outlier factor, one-class support vector machine - not stacked, but balanced through agreement. Fewer false alarms appeared when tested; 15 or so readings vanished under group judgment, leaving only what truly stood out across views. Each flagged case now carries a reason tag, thanks to SHAP tools peeling back layers behind decisions. Patterns repeated themselves - credit history, loan term length kept rising as key clues, not because someone assumed, but because numbers backed it repeatedly. What made it work smoothly? Automatic threshold tuning using interquartile range nudged settings into place without human guesswork. Scores formed in three directions gave depth, showing more than pass-or-fail labels ever could. People who handle data, even if they do not code daily, can step right in via a live dashboard built with Streamlit - no prior setup needed, no special gear, no sample answers required ahead of time. The tests show data quality hits 90.6%, while the process wraps up in about 22 seconds using regular machines, and meets every requirement laid out early on. Moving ahead, features could include handling category-based or blended data formats. Different methods like Autoencoders or DBSCAN might join the mix, adding variety to how issues are spotted. Live updates may become possible by linking tools like Apache Kafka, allowing constant oversight right inside active workflows.

REFERENCES

- [1] Aggarwal, C. C., *Outlier Analysis*, 2nd ed. New York, USA: Springer, 2017.
- [2] Arrieta, A. B., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., Chatila, R., and Herrera, F., “Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges,” *Information Fusion*, vol. 58, pp. 82–115, 2020.
- [3] Breck, E., Polyzotis, N., Roy, S., Whang, S. E., and Zinkevich, M., “Data Validation for Machine Learning,” in *Proc. MLSys*, 2019.
- [4] Breunig, M. M., Kriegel, H. P., Ng, R. T., and Sander, J., “LOF: Identifying Density-Based Local Outliers,” *ACM SIGMOD Record*, vol. 29, no. 2, pp. 93–104, 2000.
- [5] Chandola, V., Banerjee, A., and Kumar, V., “Anomaly Detection: A Survey,” *ACM Comput. Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
- [6] Cichy, V., and Rass, S., “An Overview of Data Quality Frameworks,” *IEEE Access*, vol. 7, pp. 24634–24648, 2019.
- [7] Frye, C., de Mijolla, D., Begley, T., and Goodfellow, I. J., “Shapley-Based Explainability for Anomaly Detection,” *arXiv preprint arXiv:2006.04736*, 2020.
- [8] Goldstein, M., and Uchida, S., “A Comparative Evaluation of Unsupervised Anomaly Detection Algorithms for Multivariate Data,” *PLOS ONE*, vol. 11, no. 4, 2016.
- [9] Liu, F. T., Ting, K. M., and Zhou, Z. H., “Isolation Forest,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.
- [10] Lundberg, S. M., and Lee, S. I., “A Unified Approach to Interpreting Model Predictions,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017.
- [11] Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., and Lee, S. I., “Consistent Individualised Feature Attribution for Tree Ensembles,” *arXiv preprint arXiv:1802.03888*, 2018.
- [12] Pipino, L. L., Lee, Y. W., and Wang, R. Y., “Data Quality Assessment,” *Commun. ACM*, vol. 45, no. 4, pp. 211–218, 2002.
- [13] Rahm, E., and Do, H. H., “Data Cleaning: Problems and Current Approaches,” *IEEE Data Eng. Bull.*, vol. 23, no. 4, pp. 3–13, 2000.
- [14] Redman, T. C., *Data Quality: The Field Guide*. Boston, USA: Digital Press, 2001.
- [15] Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., and Williamson, R. C., “Estimating the Support of a High-Dimensional Distribution,” *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.